

Received 5 November 2023, accepted 5 December 2023, date of publication 12 December 2023,
date of current version 19 December 2023.

Digital Object Identifier 10.1109/ACCESS.2023.3342077

RESEARCH ARTICLE

An Oracle-Based Framework for Implementing a Quantum Parallel Decoder/Multiplexer

STAVROS SOURAVLAS¹, (Member, IEEE), MARIA ORFANAKI, AND EVANGELIA GIROUSI

Department of Applied Informatics, University of Macedonia, 54636 Thessaloniki, Greece

Corresponding author: Stavros Souravlas (sourstav@uom.edu.gr)

ABSTRACT Quantum computers excel at solving complex computational problems more efficiently than conventional supercomputers. However, a significant challenge in quantum computing is implementing key operations of conventional computers, such as arithmetic operations and decoding, using quantum hardware (quantum gates). The absence of quantum gates directly implementing fundamental Boolean operations poses a hurdle. This paper introduces an oracle-based approach for implementing quantum decoding and multiplexing in a joint circuit, where oracles function as pre-programmed black boxes determining data flow. The parallel operation of oracles distinguishes this approach from sequential circuits, improving overall performance. The study validates the design using the Qiskit open-source software development kit through extensive simulations.

INDEX TERMS Quantum computing, multiplexing, decoding.

I. INTRODUCTION

Quantum computing has shown a high potential for handling and solving extremely complex computational problems in a variety of scientific fields, as a result of the natural effects of quantum superposition [1], [2]. For example, quantum algorithms can speed up classical searching schemes, by taking advantage of quantum parallelism. We already know that Grover's algorithm offers quadratic speed-up, in finding an item from an unstructured list of elements. One of the main challenges in the field of quantum computing, is the implementation of a system architecture capable of scaling to very large number of qubits. Such an architecture will be of service to address difficult problems, which can not be solved with conventional systems. One of the desirable features of such an architecture is to include circuits with less cost, in terms of the *total cost* that is, the number of gates being used and the *garbage outputs* produced (outputs which are not primary or inputs to other gates) [3], [13].

Conventional computers suffer from loss of information and energy when basic operations are performed [4]. The term *reversible circuits* [14], [16] is used to describe circuits which are information lossless. Theoretically, the

fore-mentioned loss can be as low as zero. A reversible circuit (gate) has a one-to-one mapping, between vectors of inputs and outputs; thus the vector of input states can be always reconstructed from the vector of output states. Therefore, during the computations performed in a reversible circuit, there is no information loss. The quantum circuits are typical examples of reversible computing. On the contrary, conventional known logic gates like AND, OR or XOR are not reversible -simply because they take multiple bits to just one-, so some information must be lost.

The basic unit of information in quantum computing is the qubit. The basic difference between the bit and the qubit is that a qubit can be in one *basic* state 0 or 1 as well as in any other state between 0 and 1, when the latter is found in superposition. In other words, its current state is rather probabilistic and what we are really aware of, is its probability of being in a specific state. Its representation is given by a state vector, which includes the probability of the result, that will be found when the qubit is computed. Because qubits can be in any state other than 0 or 1, the known logic gates are not suitable for quantum operations. Instead, the reversible quantum gates which act as "data path trackers", are employed for the design of quantum circuits. Such gates are the CNOT gate, the Toffoli (CCNOT gate) and the X gate, which will be used for our design.

The associate editor coordinating the review of this manuscript and approving it for publication was Shuangqing Wei².

When dealing with quantum algorithms, an important notion is the *oracle*. An oracle can be thought of as a black box that contains a number of switch statements, that is, based on the input, an oracle automatically runs an operation, that depends on the input value. In a more mathematical view, an oracle can be thought of, as a set of algebraic operations between vectors. Such a paradigm we describe in our decoding/multiplexing framework. Considering the oracle as a black box of switch statements, we realize that it has to be programmed (just like the switch statements are programmed). This programming is based on placing a set of gates, which will accomplish the task, that the oracle is designed for. Linear algebra matrix multiplications can be used to verify that the oracle, indeed performs the desired task. In the case of our decoder/multiplexer circuit oracles will be used to predict, (remember that, according to Greek mythology, people were consulting with an oracle to learn the will of the gods for their lives and their future) if a certain datum is decoded or multiplexed, based on the input.

This paper proposes a framework for the implementation of a joint $n \times 2^n$ decoder and $2^n \times 1$ multiplexer using universal Toffoli, CNOT and X gates. The three main advantages of our framework are as follows: (1) to the best of our knowledge, it is the first effort that parallelizes the decoding/multiplexing operations, through the use of programmable oracles, (2) it can be extended to any decoder/multiplexer size by simply changing the parameter of n , (3) it produces no garbage outputs (that is, all the outputs are either used as inputs to another part of the circuit or they are the basic outputs), and (4) as far as we are acknowledged, it is the first implementation of a joint quantum decoder/multiplexer. The remaining of this work is organized as follows: Section II discusses other implementations of quantum decoders or multiplexers. In Section III, we present some background knowledge on the quantum gates required to implement our framework. Section IV discusses the framework in details. Section V presents our analysis and tests in Qiskit development Kit. Finally, Section VI concludes our work and discusses possible extensions.

II. RELATED WORK

Quantum algorithms are implemented using quantum circuits. These circuits are inherently reversible in nature and contain a considerable Boolean part, that needs to be synthesized. The design of quantum circuits is a difficult task, taking into account that some conventional Boolean operations are not directly supported by quantum gates. Hence, the design of quantum circuits for various purposes is a subject that has attracted attention by researchers [16], [17], [18], [19], [20], [21], [22]. In this paragraph, we briefly describe algorithmic efforts by various researchers, in order to develop decoder or multiplexer circuits.

In [2], the authors proposed a Quantum-dot cellular automata (QCA) technology to implement a quantum

8-to-1 multiplexer. The main goal of Using the QCA technology, is to make the circuit more robust and to reduce its overall space. In [3], the authors proposed generalized algorithms for the construction of Quantum multiplexers (QMux) and demultiplexers (QDe-MUX), using the universal Fredkin gate for any given number of select lines. These algorithms were proven to be efficient, to produce valid size of Quantum multiplexers and de-multiplexers. Those two algorithms (for the multiplexer and the demultiplexer) use the fact that all circuits can be expanded logically, if logic can be derived from simultaneous constructions of circuits. For the QMux algorithm, which is the driving force of our work, the performed steps are briefly described as follows: Initially, the number of data and select lines, d and s respectively are read. Then, the number of Fredkin gates $m = d/2$ is computed for each stage. The inputs of these gates are as follows: The first input is the first select line, the second and third input are the data line values in increasing order ($D_0, D_1, D_2 \dots$). Then, the s value is reduced by one and the scheme is repeated until s reaches 0. In every iteration, m is updated as $m \leftarrow m/2$. While $s > 0$, the inputs to the Fredkin gates are as follows: the first input is the next select line and the second and third input are any two of the stored outputs of the Fredkin gates in the previous stage. The algorithm complexity is based on the value of s . The Fredkin gate based approach is the approach we have selected for our comparisons, for two reasons: (1) it is the only approach that works for any number of inputs (thus it is expandable), and (2) it is a gate-based approach, just like our strategy.

Another intriguing approach is found in [4]. The proposed circuit is an energy tolerant, low power, reversible multiplexer with optimum energy loss. Also, the garbage outputs are reduced. The idea is also gate-based and is founded on the use of the Cog gate (also known as New Fredkin gate). The Operation of cog gate is almost similar to that of Fredkin gate. In case of Fredkin gate, it generates only one Swapping operation of the control inputs. Whereas in Cog gate, it generates the two Swap operation of all control inputs. The main disadvantage of the proposed multiplexer is that it is not expandable. Its constant size is 4-to-1. From the analysis performed, the number of garbage outputs is 5, while the quantum cost is 16 (in Section V, we analytically show how to compute the quantum cost). Syamala and Tilak [5] proposed reversible multiplexer/demultiplexer circuits taking advantage of the use of reversible quantum gates. The proposed design for the multiplexer employs six gates (four Fredkin, one Toffoli and one Peres) gates in order to generate one output from four inputs depending on the selection line. Again, the proposed design suffers from input limitations. The proposed array of gates only implements a 4-to-1 multiplexer with 10 garbage outputs and a total quantum cost of 29. Similar limited in terms of inputs decoder circuits have been proposed in [13] and [14]. In [15], a compact realization of quantum n -to- 2^n decoder is proposed. The proposed design of the quantum is composed of the quantum 2-to-4 decoder circuit. The 2-to-4 decoder circuit

is implemented using Peres and CNOT quantum gates, which is used as a base to construct an $n - to - 2^n$ decoder.

This work presents an oracle based approach to implement a circuit, that can act both as a decoder and multiplexer, with distinct outputs for the two circuits. The main contributions of our work are in summary as follows:

- 1) It introduces a circuit, where all the outputs serve as inputs to other parts of the circuit, so there is no garbage inputs/outputs existed.
- 2) The oracles operate in parallel, to speedup the decoding/multiplexing operations.
- 3) Theoretical comparisons show that our proposed scheme, although necessarily requires more hardware, it operates much faster compared to other schemes.
- 4) The work has been tested for functionality using a real quantum environment. The description of our Qiskit implementation is given in Section V.

Table 1, in Section V, presents some of the results found for relatively small number of inputs, for the decoder circuit.

III. BACKGROUND KNOWLEDGE

The basic storage unit in quantum computing is the *qubit*. The general form of a qubit q is

$$q = a|0\rangle + b|1\rangle, \quad (1)$$

where $|a|^2 + |b|^2 = 1$. The basic state $|0\rangle$ is described by the matrix $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ which states that the probability of the qubit to be equal to 0 is certain (marked as 1) and the probability of the qubit to be equal to 1 cannot happen (zero possibilities).

Similarly, the basic state $|1\rangle$ is described by the matrix $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ which states that the probability of the qubit to be equal to 0 is impossible and the probability of the qubit to be equal to 1 is the only possible outcome. There are more basic states for the qubit like the $|+\rangle$ or the $|-\rangle$ state, but we will not deal with these states in this work.

When more qubits are involved, the state that is produced is determined by the tensor product of the individual qubit state. For example, a 2-qubit register that contains two qubits in state 0, is:

$$|00\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

where the respective probabilities are 1 for the qubits to be in state $|00\rangle$, and for all the other states $|01\rangle$, $|10\rangle$, $|11\rangle$ it is 0.

Below, we describe the basic gates that we use for our implementation: The Hadamard gate, the X-Pauli gate, the CNOT(cX) gate, and the CCNOT(ccX or Toffoli) gate.

A. THE X-PAULI GATE

The Pauli gates are the three Pauli matrices (σ_x , σ_y , σ_z) and have a direct effect on a single qubit. The Pauli X, Y and Z

associate, proportionately, to a rotation around the X, Y and Z axes of the Bloch sphere by π radians [23].

The classical NOT gate, is called the Pauli-X Gate or simply X-Gate because it rotates the vector 180 degrees around the X-axis. The symbol of the X-gate is $\boxtimes$. It is sometimes called a bit-flip as it maps $|0\rangle$ to $|1\rangle$ and $|1\rangle$ to $|0\rangle$ [23]. The matrix of the Pauli-X Gate is represented as: $X = \sigma_x = NOT = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ [23] and its multiplication by the qubit $|q\rangle = \begin{pmatrix} a \\ b \end{pmatrix}$ is as follows:

$$X \cdot |q\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 0+b \\ a+0 \end{pmatrix} = \begin{pmatrix} b \\ a \end{pmatrix}$$

B. HADAMARD GATE

One of the most important gates that distinguish classical from quantum computers is the Hadamard Gate (its symbol is $\boxplus$), or simply H-Gate. The H-Gate rotates qubits' direction on Bloch Sphere 180 degree around the vector that lies equidistantly between the X and Z axes. Thus, it is defined as: $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$.

This gate can convert a classical situation into a superposition:

$$H|q\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} a+b \\ a-b \end{pmatrix}$$

Specifically, if we multiply it either by the $|0\rangle$ state or the $|1\rangle$ state, the product gives 0.5 chance of being counted as $|0\rangle$ and 0.5 as $|1\rangle$. The product with $|0\rangle$ gives a special state which we label plus state and symbolize as $|+\rangle$, while $|1\rangle$ gives a special state which we refer to as the minus state and symbolize it as $|-\rangle$.

C. THE CONTROLLED-NOT (CNOT) GATE

The CNOT gate, or commonly known as controlled-x gate, is a principal quantum gate not only used in quantum computing but in classical reversible computing as well. It is applied on two qubits; one acting as 'control' and the other as 'target'. The CNOT gate leaves the target qubit unaltered when the control qubit is in state $|0\rangle$ and as for the control qubit it leaves it unchanged and performs a Pauli-X gate on the target one when the control qubit is in state $|1\rangle$ [24]. The CNOT gate can be represented by the following matrix:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Suppose someone is in the situation as described below:

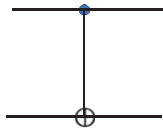
$$|10\rangle = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

The control qubit equals to 1 and the target one equals to 0. If we multiply it by the representation of the gate, the result we are going to achieve

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |11\rangle$$

So the qubit target has been reversed.

The graphical symbol of the CNOT gate is



where the $\bullet$ is the control qubit and $\oplus$ is the target qubit.

If $|0\rangle$, $|1\rangle$ are the only permitted values for both qubits as input, then the output of the *CNOT* gate is correlated with the outcome of the *XOR* gate. Taking into account that the control qubit is $|1\rangle$, the target output of the *CNOT* gate corresponds to the result of the *NOT* gate. In general, the inputs are allowed to be a linear superposition of $|0\rangle$, $|1\rangle$ [25]. The *CNOT* gate is able to convert the quantum state:

$$a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle \rightarrow a|00\rangle + b|01\rangle + c|11\rangle + d|10\rangle \quad [25]$$

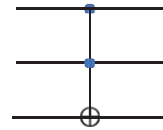
Constructing a function-controlled *NOT* gate, which accepts $n + 1$ qubits as input, where $2 \leq n + 1$ (a quantum register) is attainable. This gate reverses the last qubit of the register in the case that we consider a function, with the first n qubits as input, that returns a 1 [25]. The *CNOT* gate has a cornerstone part in building quantum circuits and is especially of use if the control qubit is in super position for creating entanglement and performing operations on multiple qubits concurrently [26].

D. THE TOFFOLI - CCNOT GATE

In logic circuits, the *CCNOT*-Toffoli gate (is named after Tommaso Toffoli) is an extension of the *CNOT* gate and is a reversible gate used in quantum computing. This has the meaning that any classical reversible circuit can be formulated by *CCNOT* gates. It is often called a “controlled-controlled NOT gate” (*CCNOT* gate) [27]. A logic gate $f(x) = y$ expresses that for any output y , there is a distinctive input x and is reversible in case there is another gate $f'(y) = x$ which connects y to x . For example, the logic *AND* gate is not reversible, because the inputs 00, 01 and 10 have identical outputs, 0. On the other hand, the logic *NOT* gate is reversible because it inverts the input. The Toffoli-*CCNOT* gate is a *CNOT* gate with two control qubits and a target one. If the first two qubits (control qubits) are both set to 1, the third qubit (target qubit) will be reversed. In all other instances, all qubits remain the same. For one input bit, there are two feasible reversible gates.

The first one is the identity gate, which leads its input to the output unaltered. The second one is the *NOT* gate. For two input bits, the only significant gate is the controlled *NOT* gate, which makes use of the *XOR* gate from the first to the second bit. As a result, first bit remains the same [27].

The graphical symbol of the *CCNOT* gate is



and, as in the *CNOT* gate, the $\bullet$ are the control qubits and $\oplus$ is the target qubit.

The *CCNOT* gate, proposed in 1980 by Toffoli is a gate that has 3-bit inputs and outputs. If the first two bits are set, it inverts the third bit. The following 8×8 matrix shown below is for the Toffoli gate:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Suppose we are in the situation where

$$|110\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

The multiplication will be:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = |111\rangle$$

It can also be expressed by connecting bits: $a \rightarrow a$, $b \rightarrow b$ and $c \rightarrow c \text{ XOR } (a \text{ AND } b)$ which equals to the modulo operation $c \rightarrow (c + ab) \bmod 2$ or $c \rightarrow c \oplus ab$ [27].

E. CONVENTIONAL DECODER/MULTIPLEXER

The multiplexer behavior resembles to a large extent to the behavior of the decoder. A $2^n \times 1$ multiplexer includes a $n \times 2^n$ decoder plus n data input lines. Fig. 1 shows this situation for $n = 3$. The selection lines s_2, s_1, s_0 are inputs to a 3×8 decoder. Each combination of inputs activates one

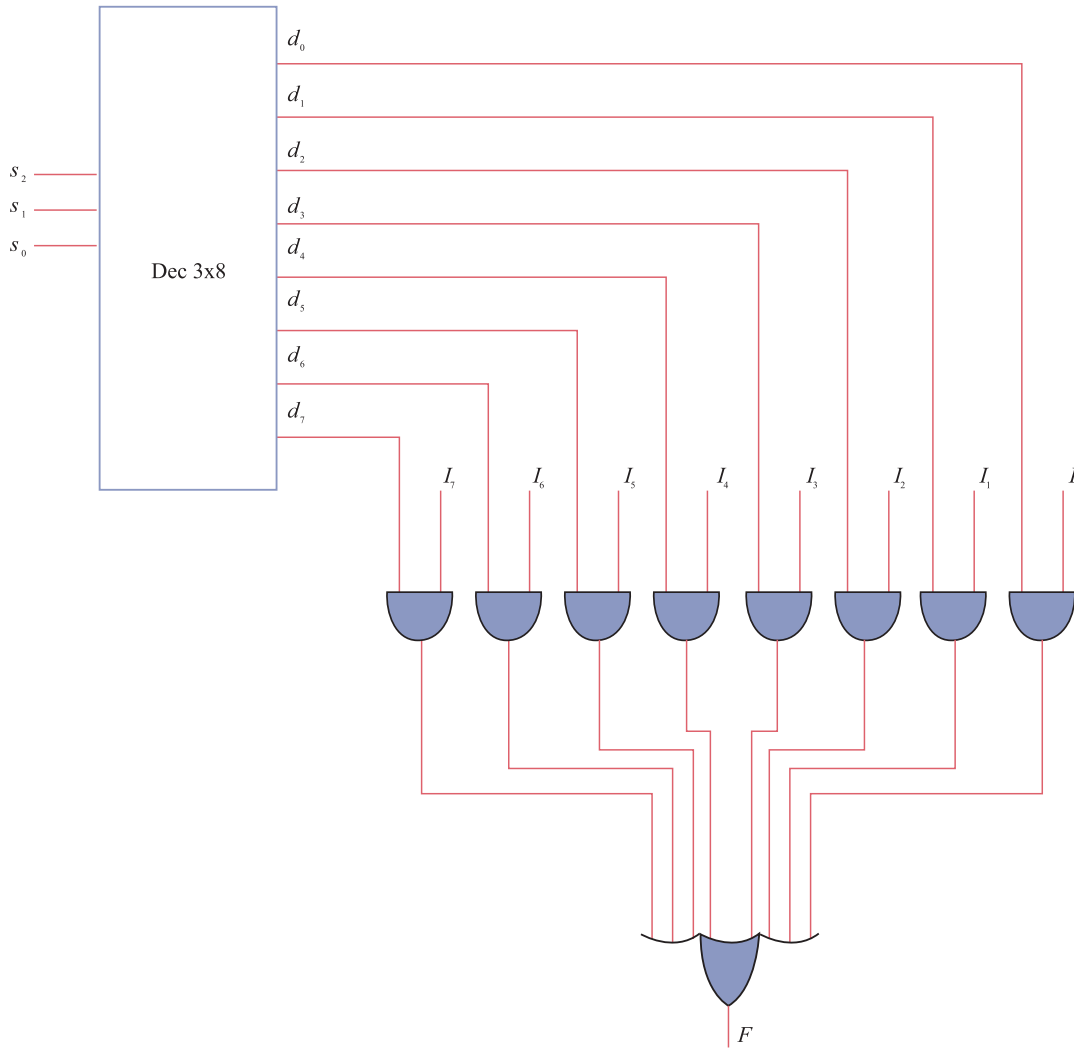


FIGURE 1. A 3×8 decoder 8×1 multiplexer.

of the decoder outputs d_0 to d_7 . Each d_j inputs a 2-input AND gate along with the data input I_j . Since only one d_j can be 1 at a time, the final output F is driven strictly by I_j . This is the idea that we try to implement in this work, under the restrictions posed by the fact that certain conventional logic operations are not directly supported by quantum gates. Another restriction is that the quantum gates operate on few only qubits, thus when we need to perform sequential logical operations on more qubits, we need to do this efficiently. In this work, we introduce certain levels of parallelism to handle these cases, as described in Section IV.

IV. OUR FRAMEWORK FOR DECODING/MULTIPLEXING

Our framework is based on the idea of programming 2^n oracles in order to implement an $n \times 2^n$ decoder - $2^n \times 1$ multiplexer. In this section, we describe our implementation for $n = 3$ and provide the necessary changes for extensions to larger circuits. In the first subsection,

we describe the decoding part, in the second subsection we describe the multiplexing part and in the last subsection we put things together.

A. DECODING THE INPUTS

Let us initially define the value of n and we start with an n -bit quantum register. As stated in the beginning of this section, for our description, we set $n = 3$. The initial state of this register is $|000\rangle$. Algebraically, this is expressed as the tensor product of the three zero-state matrices:

$$|000\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad (2)$$

Next, let us apply the Hadamard gate on the input register. This is denoted by the expression $H^{\otimes n}$. To do so, we first compute the tensor product of three matrices that define the Hadamard gate and then we apply a matrix multiplication between this tensor product and the matrix in Eq. 2:

$$\begin{aligned} H^{\otimes 3} &= H \otimes H \otimes H = \left[\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \right]^2 \otimes H \\ &= \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \\ &= \frac{1}{2\sqrt{2}} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \end{pmatrix} \end{aligned} \quad (3)$$

Then by applying matrix multiplication between the resulting matrix from Eq.3 and the matrix from Eq.2, we obtain:

$$H^{\otimes 3}|000\rangle = \frac{1}{2\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \quad (4)$$

The result of Eq.4 can be rewritten as

$$H^{\otimes 3}|000\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle) \quad (5)$$

From Eq.5 we note that the application of a single Hadamard gate on our 3-qubit input provides us with all the possible basic qubit states (in this case, $2^3 = 8$). Each state has a probability of $(\frac{1}{2\sqrt{2}})^2 = \frac{1}{8}$.

1) THE ORACLES OF OUR FRAMEWORK

The basic idea behind the oracles is to program them in such a manner that each oracle is “predicting” the “good fate” of a certain datum, that is, whether it will be selected or not.

Each oracle $\mathcal{O}_j$ ($j \in [0, \dots, 2^{n-1}]$) has n inputs S_{n-1} to S_0 , $n-1$ qubits C_{n-2} to C_0 used for the necessary computations within the oracle (set as 0 initially) and I_j , the datum to be selected. Also, it has two output qubits, d_j (the decoder output) and G_j , the output to be OR-ed in order to provide the final output of the multiplexer. Each oracle (see Fig. 2) $\mathcal{O}_j$ accepts the 3 input qubits S_{n-1} to S_0 and

predicts favorably for datum I_j only for one combination of these inputs. Proposition 1 is used to prove this fact.

Proposition 1: The oracles of our framework are organized in such a manner each one of them selects as output a different data element, based on a different input combination (decoding).

Proof: First, let us define a function $F(S) = C_0$, where S defines all the possible combinations of the input qubits and C_0 is the decoder final output produced by each oracle (see Fig. 2). Then,

$$\begin{aligned} F(S) &= \text{CCNOT}(S_2, C_1, C_0) \\ &= \text{CCNOT}(S_2, \text{CCNOT}(S_0, S_1, C_1), C_0) \end{aligned} \quad (6)$$

Let us start from $\mathcal{O}_0$. As described in Eq.5 the application of a Hadamard on $S_2 - S_0$ provides 8 combinations. We need to analyze all 8 of them.

Case 1: $S_2, S_1, S_0 = 000$: The X gates will change the 3 qubits to 111. Then, the first CCNOT gate is applied on C_1 as target (initially C_1 is set to 0), with control qubits S_0 and S_1 . The result can be algebraically expressed as:

$$\begin{aligned} \text{CCNOT}|S_0S_1C_1\rangle &= \text{CCNOT}|110\rangle \\ &= \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} \\ &= \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = |111\rangle \end{aligned}$$

Proceeding similarly, the second CCNOT will be applied on target C_0 (initially set to 0) and control qubits S_2 and C_1 . The value of C_0 will be one of the 8 decoder values. Again, it will be

$$\text{CCNOT}|S_2C_1C_0\rangle = \text{CCNOT}|110\rangle$$

which, as before equals to $|111\rangle$. Thus, $C_0 = 1$.

Case 2: $S_2, S_1, S_0 = 001$: The X gates will change the inputs to 110. Proceeding as in Case 1,

$$\text{CCNOT}|S_0S_1C_1\rangle = \text{CCNOT}|110\rangle = |111\rangle$$

and

$$\text{CCNOT}|S_2C_1C_0\rangle = \text{CCNOT}|010\rangle = |010\rangle$$

to make $C_0 = 0$.

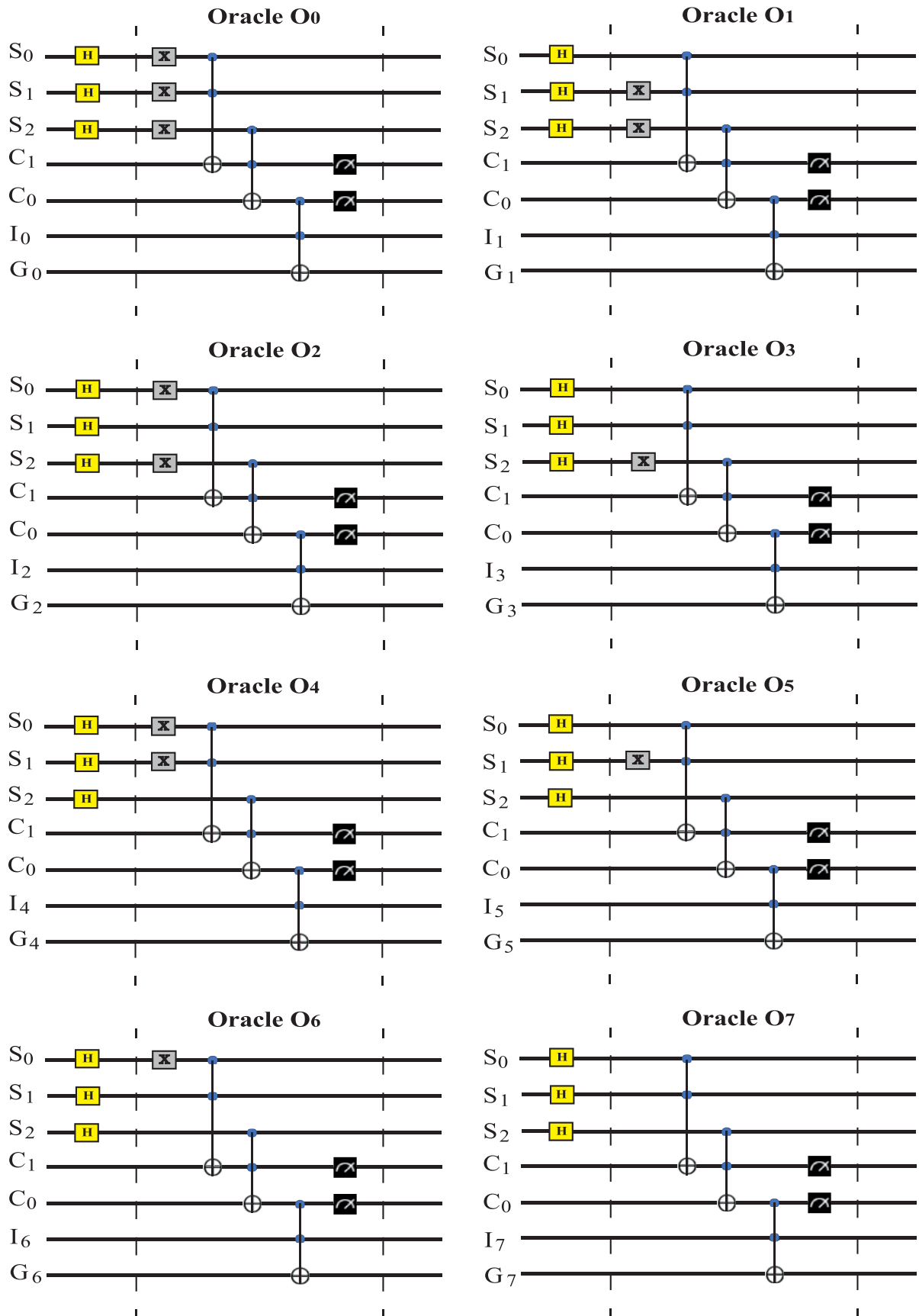


FIGURE 2. Oracles for a 3×8 decoder - 8×1 multiplexer.

Cases 2-8: For the other combinations of $S_2 - S_0$, 010, 011, 100, 101, 110, and 111, with similar computations we find that $C_0 = 0$.

From the analysis performed, we can infer that C_0 , one of the decoder outputs produced by oracle $\mathcal{O}_0$ will be 1 only when the two CCNOT operations required to compute $F(x)$ (see Eq. 6) are of the form

$$\text{CCNOT } |110\rangle \quad (7)$$

The same combination of inputs to the CCNOT gate is required for all the other oracles, but this appears for different inputs combinations. For example, in oracle $\mathcal{O}_2$, the operation of Eq. 7 will appear for inputs $(S_2, S_1, S_0) = 010$. ■

Proposition 2 proves mathematically the statement of Proposition 1.

Proposition 2: Each oracle decodes only one input combination with probability 1. The probability of decoding one of the remaining input combinations is 0.

Proof: Now, let us define G as a function that computes the sum of the results of $F(S)$ for all the possible combinations of n inputs.

$$G = \left(\sqrt{2}\right)^n \sum F(S), \quad S \in \{0, 1\}^n$$

We can use Eq. 5 to find $\sum F(S)$ (for $n = 3$):

$$\begin{aligned} \sum F(S) &= \frac{1}{2\sqrt{2}} (F(S)|000\rangle + F(S)|001\rangle + F(S)|010\rangle \\ &\quad + F(S)|011\rangle + F(S)|100\rangle + F(S)|101\rangle \\ &\quad + F(S)|110\rangle + F(S)|111\rangle) \end{aligned} \quad (8)$$

Thus, for a specific oracle, say $\mathcal{O}_3$, Eq. 8 will produce:

$$\begin{aligned} \sum F(S) &= \frac{1}{2\sqrt{2}} (0 \times |000\rangle + 0 \times |001\rangle + 0 \times |010\rangle \\ &\quad + 1 \times |011\rangle + 0 \times |100\rangle + 0 \times |101\rangle \\ &\quad + 0 \times |110\rangle + 0 \times |111\rangle) \\ &= \frac{1}{2\sqrt{2}} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \end{aligned} \quad (9)$$

The term $\left(\sqrt{2}\right)^n$ in our computation of G has been introduced to serve the purpose of removing the probability factor introduced by the Hadamard gates. Thus:

$$G = \left(\sqrt{2}\right)^3 \frac{1}{2\sqrt{2}} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} = |011\rangle$$

which indicated that oracle $\mathcal{O}_3$ decodes only the state $|011\rangle$ and selects the datum associated to this state.

The same analysis for the remaining oracles similarly reveals that $\mathcal{O}_1$ decodes only state $|000\rangle$, $\mathcal{O}_2$ decodes only state $|010\rangle$, ..., and $\mathcal{O}_7$ decodes only state $|111\rangle$. This completes the proof. ■

B. SELECTING THE OUTPUT

The oracle that controls input selection is shown in Fig. 3. In the previous section, we proved that each oracle can only decode one input combination, that is, the C_0 value of each oracle will be equal to 1 just for the combination the oracle is programmed to select. Thus, in any oracle $\mathcal{O}_j$ the last CCNOT operation that has qubit G_j (initially set to 0) as target will be:

$$\begin{aligned} G_i &= \text{CCNOT}(C_0, I_j, G_j) \\ &= \text{CCNOT}(1, I_j, 0) = I_j \end{aligned} \quad (10)$$

provided that the combination of $S_{n-1}, \dots, S_0$ will be such that $C_0 = 1$. Otherwise, $G_j = 0$. Thus, the value of output G_j for each oracle will follow the data input qubit I_j . These G_j outputs are inputs to the oracle that performs the overall data selection. From the analysis in the previous subsection, it is obvious that for one given combination of inputs $S_{n-1}, \dots, S_0$, there will be only one G_j value (from the oracle that decodes the combination) that is driven by an input I_j : the remaining G_j values will be 0. Therefore, output F (initially set to 0) will be computed by

$$\text{CNOT}(G_i, 0) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = |00\rangle$$

for as long as G_i 's remain 0. Once we reach the G_j value that is driven by I_j then, if $I_j = 0$ then the computations follow the same pattern. If $I_j = 1$, then we will have:

$$\text{CNOT}(G_i, 0) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = |11\rangle$$

which means that $F = 1$. If F changes to 1, it will not go back to 0, because the rest of the CNOT operations will involve G_j values of 0:

$$\begin{aligned} \text{CNOT}(G_i, F) &= \text{CNOT}(0, 1) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \\ &= \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = |01\rangle \end{aligned}$$

meaning that F remains 1. Notice that we have broken the 2^n OR operations into two parallel sets. To do so, we introduce

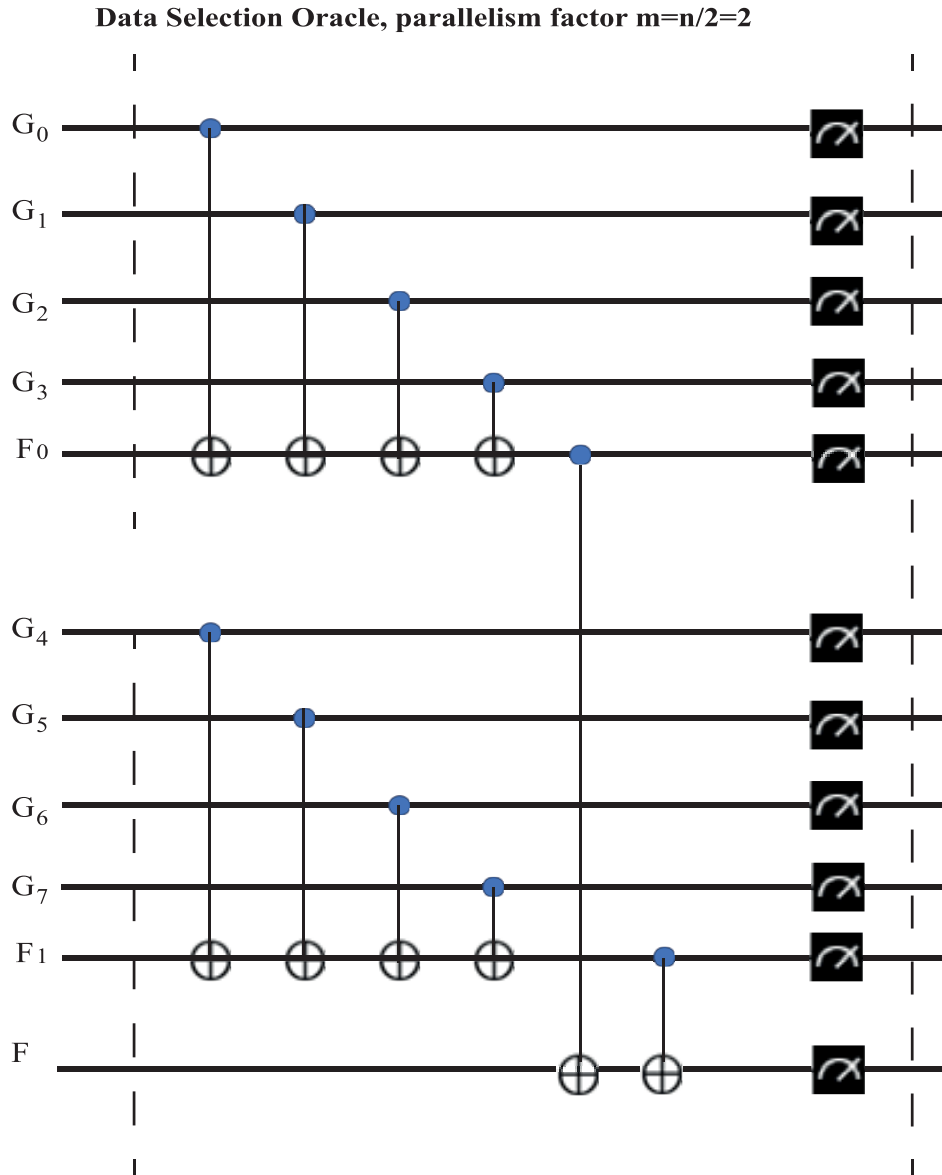


FIGURE 3. Oracle for final data selection.

the *parallelism factor* m as

$$m = \lceil \frac{n}{2} \rceil \quad (11)$$

The m factor indicates that when our circuit needs to perform 2^n OR operations in the end, it parallelizes groups of 2^m of them (thus obtaining $\frac{2^n}{2^m}$ partial OR results, which are then executed sequentially (or they are parallelized using a new m' value, $m' = \frac{m}{2}$). In our example, we need to perform 8 OR operations. Assuming that each has a delay of 1 unit of time, their total delay is 8. However, by using $m = 2$, we parallelize groups of $2^m = 4$ of them and then we execute sequentially $\frac{2^n}{2^m} = 2$ ORs (the 2 partial results which are Ored to get the final result). This approach gives a delay of $4+2 = 6$ time units. This is shown in Fig. 3. To see the benefit of this approach, suppose that $n = 9$

(9×512 decoder/ 512×1 multiplexer). We have $m = 5$, meaning that we organize the 512 OR operations into groups of $2^5 = 32$ parallel operations which cost 32 and produce $\frac{2^9}{2^5} = 16$ partial OR results that can be executed sequentially. This will give a delay of $32+16 = 48$ time units, which is about 10 times faster compared to the cost of 512 that would incur if we executed these operations sequentially.

C. PUTTING THINGS ALTOGETHER

Fig. 4 shows how the quantum decoder/multiplexer operates. Upon arrival of an input, all the oracles “predict” if the input is decoded. Simply, the one that is able to decode is the one that outputs the G_j value that is guided by the input qubit I_j . The data selection oracle accepts the G_i inputs ($n - 1$ zeros and one driven by I_j) and produces the output F . For example,

if $s_2, s_1, s_0 = 0, 1, 0$, then G_2 is guided by I_2 and the remaining G_j values are zero. Thus, the data selection oracle will produce an F output equal to I_2 .

V. ANALYSIS AND QISKIT SIMULATIONS

In this work, we have proposed an oracle based parallel decoder/multiplexer. We use the regular performance metrics to evaluate our circuit.

The metrics that we use to measure the performance of our circuit is the *delay* of the logical calculations combined to the overall quantum cost and the number of *garbage outputs*.

A. CIRCUIT QUANTUM COST

The quantum costs of all the reversible 1×1 and 2×2 quantum gates is computed by counting the numbers of NOT, Controlled-V, Controlled-V+ and CNOT gates used. For the gates we use in this work, the Toffoli (CCNOT) gate has a cost of 5 (2 V- gates, one V+ and 2 CCNOT). Obviously, the X gate and the CNOT gates have a cost of 1). The quantum cost QC of a circuit is defined as the sum of the costs of all the gates it involves:

$$QC = \sum_{k=1}^{\ell} p_k \times c_k \quad (12)$$

where ℓ is the number of different gates used by the circuit, p_k is the number of gates of a specific type and c_k is the cost of a gate of type k . In our circuit, we have $\ell = 3$ (X, CNOT, CCNOT). Thus, if we set as c_1 is the cost of an X gate, c_2 is the cost of an CNOT gate and c_3 is the cost of an CCNOT gate, then $c_1 = 1$, $c_2 = 1$, and $c_3 = 5$. Also, we have $c_1 = 2^n \times \frac{n}{2}$ X gates (notice that if we take all the input combinations, half of the input qubits are complemented), $p_2 = 2^n$ CNOT gates and $p_3 = 2^n \times n$ CCNOT gates. Thus, the total number of gates is

$$\begin{aligned} \text{No. of gates} &= 2^n \times \frac{n}{2} + 2^n + 2^n \times n = 2^n \left(\frac{n}{2} + n + 1 \right) \\ &= 2^n \left(\frac{3n}{2} + 1 \right) \end{aligned} \quad (13)$$

The total quantum cost for our circuit will be

$$\begin{aligned} QC &= (2^n \times \frac{n}{2}) \times 1 + (2^n \times n) \times 5 + 2^n \times 1 \\ &= 2^n + 2^n \frac{n}{2} + 5n2^n \end{aligned} \quad (14)$$

As an example, the 3×8 decoder / 8×1 multiplexer described in Figures 2, 3, 4 will have

$$\text{No. of gates} = 2^3 \left(\frac{3 \times 3}{2} + 1 \right) = 44$$

and its quantum cost will be

$$QC = 2^3 + 2^3 \times \frac{3}{2} + 5 \times 3 \times 2^3 = 8 + 12 + 120 = 140$$

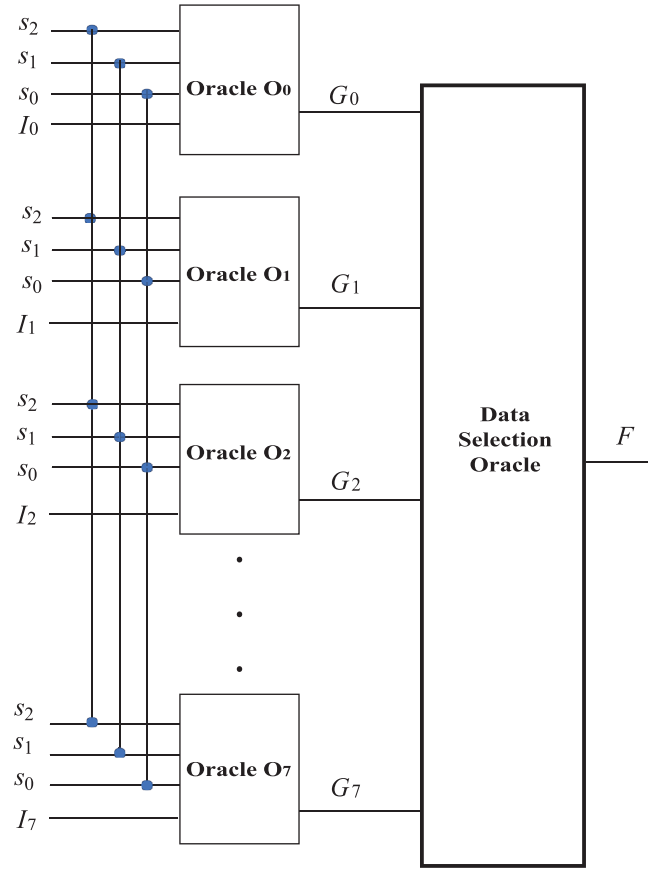


FIGURE 4. General organization of our framework.

B. DELAY ANALYSIS AND COMPARISONS

The delay metric is based on the number of logical calculations that the circuit performs. To compute the delay of the logical calculations, we define:

- α = the number of XOR calculations
- β = the number of AND calculations
- c = the number of NOT calculations
- d = the number of OR calculations

The delay of a logic circuit is the maximum number of gates in a path from any input line to any output line. The definition is based on two assumptions: (i) Each quantum gate performs computation in one unit time and (ii) all inputs to the circuit are available before the computation begins [22]. To analyze the delay of our circuit, we have to consider that the inputs are made available to each oracle before the computation begins. Let us find the delay of each of the decoding oracles $\mathcal{O}_{n-1}$ to $\mathcal{O}_0$. These oracles perform from 0 to 2^n X (see Fig. 2) operations in parallel, thus their cost is just 1 unit time. Also, each oracle makes n sequential AND computations (performed by the CCNOT gates) to find the G_j values. Among the oracles, these computations are executed in parallel, so their cost is n . Finally, the data selection oracle performs 2^n CNOT operations to obtain the final output F .

TABLE 1. Comparison of our scheme to the Fredlin gate based approach multiplexer [3] and the reversible quantum decoder [15].

Approaches	n	Circuit Size	Number of Gates	Number of Garbage Outputs	Quantum Cost	Parallelism Factor m	Delay
2^n -to-1 MUX [3]	1	2×1	1	2	5	—	8
	2	4×1	3	5	15	—	24
	3	8×1	7	10	35	—	56
	4	16×1	15	19	75	—	120
	...	...	...	...	...	—	...
	9	512×1	511	519	2555	—	4088
	...	...	...	...	...	—	...
	n	$2^n \times 1$	$2^n - 1$	$2^n + \log_2(n) - 1$	$5(2^n - 1)$	—	$(2^n - 1) \times 8$
n -to- 2^n DEC [15]	2	2×4	7	0	19	—	7
	3	3×8	27	1	65	—	28
	4	4×16	61	4	262	—	61
	...	...	...	...	...	—	...
	n	$n \times 2^n$	$[7 \times (3 \times \lceil 2^n - 5 \rceil + 5 \times (2^n/2))]$	—	$A^* \times [3 \times \lceil 2^n - 5 \rceil + 2^n/2]$	—	Same as num. of gates
Our approach	1	2×1	5	0	13	—	4
	2	4×1	16	0	48	1	7
	3	8×1	44	0	140	2	10
	4	16×1	112	0	338	2	13
	...	...	...	...	...	...	...
	9	512×1	7424	0	25856	5	58
	...	...	...	...	...	...	...
	n	$2^n \times 1$	$2^n (\frac{3n}{2} + 1)$	0	$2^n + 2^n \frac{n}{2} + 5n2^n$	$\frac{n}{2}$	$\frac{2^n}{2^m} + 2^m + n + 1$

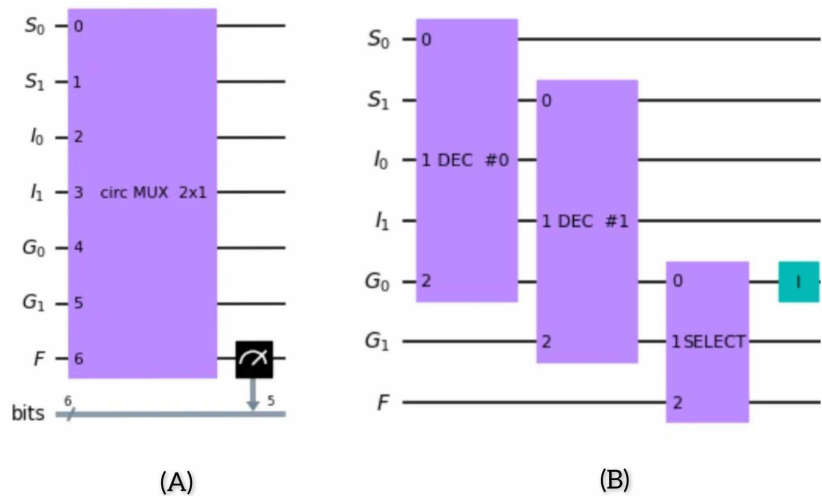


FIGURE 5. Image (A) depicts all registers that were used to code the Multiplexer 2×1 circuit. Image (B) focuses on the inside of oracle (A), and illustrates all Decoder oracles, followed by and an oracle called “Select”, in which takes part in the process of selection.

This means that the overall delay t_d will be

$$t_d = 1 + n + 2^n \quad (15)$$

Table 1 gives the delay for a number of different parameters and compares to other schemes with known delay times like [3] and [15] (the A parameter mentioned with an $*$ corresponds to the sum of $(3\Omega + 4)$, where Ω is the controlled V^+ gate cost and σ is the cost of the CNOT gate):

C. NUMBER OF GARBAGE OUTPUTS

Since the proposed circuit performs various operations in parallel, it necessarily requires more quantum gates. However, the outputs of our circuit are all either final or

they serve as inputs to other gates. This is obvious from Figures 2, 3, and 4. Thus, one of the advantages of this work is that the gate outputs serve as inputs to later other parts or later stages of the circuit.

D. COMPARISONS

From Table 1, we can see that our scheme has the following advantages. First, there are no unused outputs, in the sense that they are either final outputs or they feed other parts of the circuit. Clearly, our scheme requires more gates. However, the comparisons show that the increase of hardware pays off, by minimizing the overall delay. For example, consider a 16×1 multiplexer implemented by [3],

a 4×16 decoder implemented by [15], and a 4×16 decoder- 16×1 multiplexer performed by our scheme. Our scheme requires $\frac{112}{15} = 7.46$ times more gates compared to [3] and $\frac{112}{61} = 1.83$ times more gates compared to [15]. However, it performs multiplexing $\frac{120}{13} = 9.23$ times faster compared to [3] and decoding $\frac{61}{13} = 4.69$ times faster compared to [15]. These differences in speed increase as we increase the number of inputs. Also, take into account that our circuit performs both decoding/multiplexing in one circuit.

E. QISKIT IMPLEMENTATION

To simulate the proposed structure of a Multiplexer 8×1 , experiments were conducted using the Qiskit framework. When parameterized, we consider a Multiplexer $N \times 1$, that always applies to $N = 2^n$.

As seen in Figure 5, we aimed to create an oracle-based circuit, which is an implementation of a Multiplexer 2×1 . This circuit consists of two Decoders and one Selection box. As a rule, a Multiplexer $N \times 1$ contains N number of Decoders, and after those 1 oracle to illustrate Selection.

Initially, we declared the conventional registers by defining their purpose and the size of the qubits each Multiplexer $N \times 1$ requires, as it can be seen in Code 1. It is essential to denote, that the number of S qubits is equal to $N \times \log_2 N$, instead of $\log_2 N$, due to the fact that during Qiskit simulation all the decoders needed to receive the input value simultaneously. Time and hardware required to implement Multiplexer's input initialization, should not be taken under consideration neither as a part of the main circuit's complexity nor as additional endeavour to execute.

To be more precise, apart from the main Multiplexer circuit seen in image (A) of Figure 5, when observing Figure 6 we see another oracle before that. This sub-circuit initializes all selection qubits, labeled as S , to carry the demanded input value. The aforesaid oracle sets the first $\log_2 N$ qubits marked as S to the given input and transmits the combination to the rest of the $(N - 1) \times \log_2 N$ copies.

```
1. def standard_MUX_registers (N):
2.     S = QuantumRegister(N*log2(N),
3.         "S")
4.     C = QuantumRegister(N*(log2(N)-1),
5.         "C")
6.     I = QuantumRegister(N, "I")
7.     G = QuantumRegister(N, "G")
8.     F = QuantumRegister(numGroups+1,
9.         name = "F")
10.    bits = ClassicalRegister
11.        (log2(N)+N+1, name = "bits")
12.    return S, C, I, G, F, bits
```

LISTING 1: Standard MUX registers.

For simplicity reasons, the term of the *AND* gate is associated to an order of quantum gates, that result in

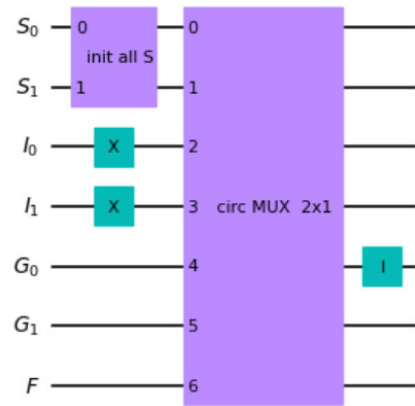


FIGURE 6. Complete version of multiplexer 2×1 , using an oracle to initialize the S values. In this example, all the qubits labeled as I are enabled.

the output 1, if and only if all the qubits involved are measured as 1. Additionally, the term of the *OR* gate is related to a succession of quantum gates, that lead to the output 1, when at least one of the input qubits is observed to be 1.

In Code 2, the function named *all_decoders()*, creates N decoders (where each decoder maps to a combination of $n = \log_2 N$ bits). Each decoder, has as input n qubit lines, being referred as S . Those should go through simulation of the *AND* gate. To do so, $n - 1$ more qubits have been used, termed as C . The last step is to take into consideration the value of the Enable line, termed as I , and the result of the *AND* gate which is stored in C_{n-2} . If both are measured as one, the Decoder's output, being represented as the G qubit, also flips from zero to one.

```
1. def create_standard_MUX_circ(N):
2.     S,C,I,G,F,cr =
3.         standard_MUX_registers(N)
4.     main = QuantumCircuit(S, C, I, G, F)
5.     DECs, lst = all_decoders(N, S, C, I, G)
6.     main.append(DECs, lst)
7.     SELECT_inst, lst = select_Gs(N, G, F)
8.     main.append(SELECT_inst, lst)
9.
10.    main.id(G[0])
11.    inst = main.to_instruction(
12.        label = "MUXcirc")
13.    return main, inst
```

LISTING 2: Create_standard_MUX_circ.

To illustrate this, in Figure 7 someone can easily observe a part of the Multiplexer 16×1 , which represents the Decoder #13. There are four input lines S , and three more, C , to fulfill the *AND* gate. The initial value of the S register is 1100 (in other words #12), and the Enable qubit, I , is set to 1. In the end, the output value, stored in G_{13} , shall be 0, because Decoder's value doesn't match the selected input.

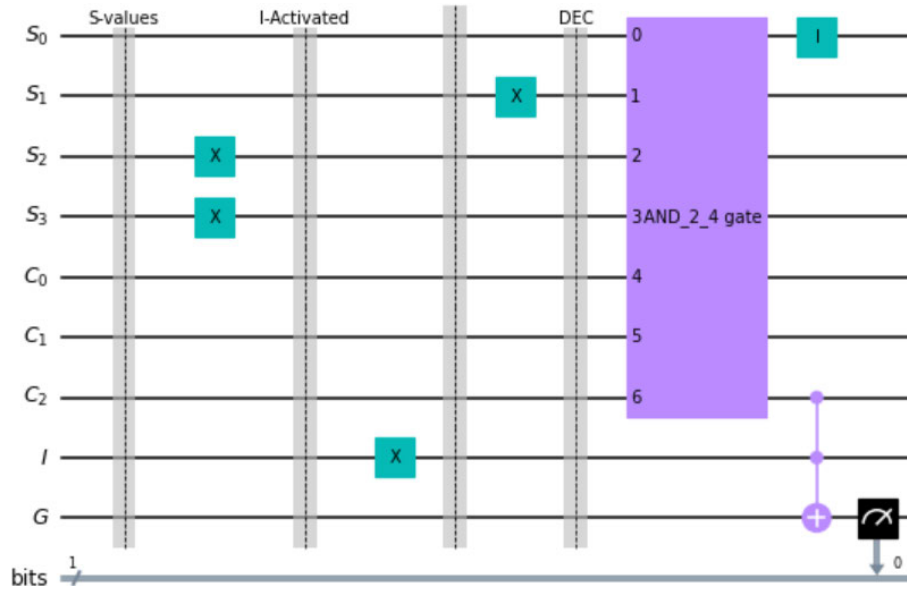


FIGURE 7. Example of a decoder frame (DEC No.1101), which has four input lines (S qubits, set on 1100) and an active enable (I qubit, set to 1). The qubit G maintains the decoder's output value.

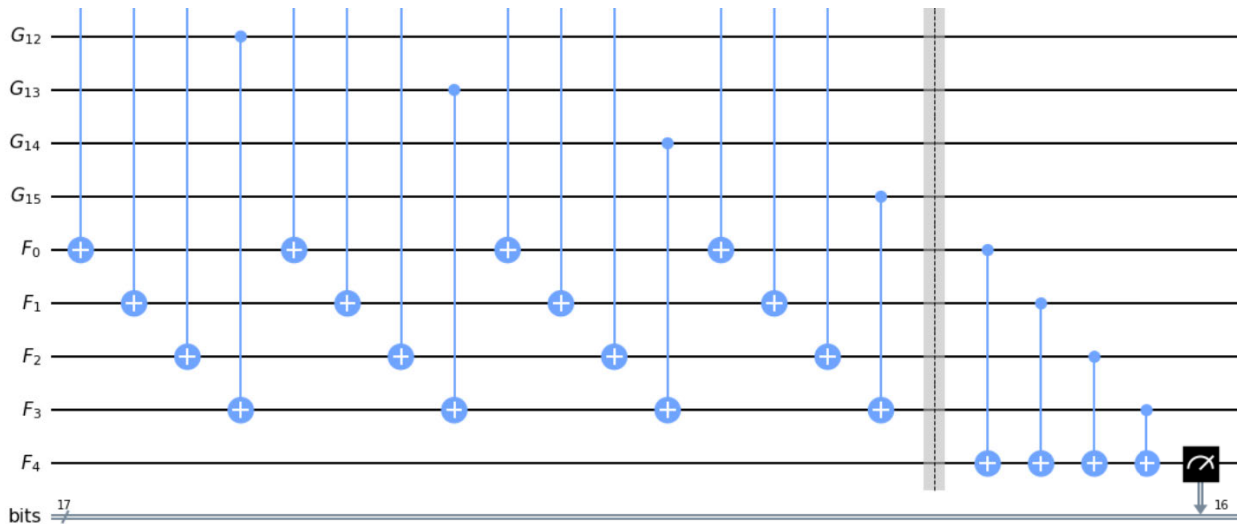


FIGURE 8. Decoders' outputs (G qubits) going through a selection process, in the case of MUX 16×1 .

In the example above, if the Decoder's output G_{13} was demanded to be equal to the ace value after moving through the Decoder No.1101 (#13), the S register had to be initialized to 1101 (#13).

On the outside of all Decoder oracles, the outputs being referred to as G qubits, transit through the process of selection. While observing Code 3, it is important to denote that each one of the G qubits is connected to one F line passing through a Controlled-NOT gate. Next, supposing the F register has r number of qubits, it can be seen that all qubits between F_0 and F_{r-2} are in Control of the qubit F_{r-1} . The latter, F_{r-1} , enfolds the Multiplexer's output value. This makes an parallel progressing of G 's. The reason why this

is working as simply as that, lye's to the fact that only one or none of the G qubits in Control is able to carry the ace value.

Furthermore, Figure 8 provides a visual representation of how one ore none, of the G 's qubits is selected. From the variety of the N Decoders' outputs, the selected value on the final F qubit is aimed to be stored. In the case of the Multiplex 16×1 , all the 16 qubit lines are obliged to go through an OR gate. In order to gain speed advantage, they are sorted in groups (with the use of the parallelization factor, m , as mentioned previously in the article). As an example, in Multiplexer 16×1 , is calculated, that we need 4 groups in qubits F_0 to F_4 . It also happens that each group

```

1. m = int(math.ceil(log2_N/2))
2. Gs_in_group = pow(2,m)
3. num_of_groups = int(N/Gs_in_group)

4. qc = QuantumCircuit(G, F, cr, name="Select")

5. for i in range(num_of_groups):
6.     for k in range(Gs_in_group):
7.         qc.cx(G[k + i*Gs_in_group], F[i])

8. for i in range(F.size-1):
9.     qc.cx(F[i], F[F.size-1])

```

LISTING 3: Selection_circ.

uses 4 of G_i qubits. On the inner side of the OR oracle a process of selecting at least one G_i , which has the value of 1, takes place.

In order to achieve that, if the number of groups is equal to k , we need a total of $k + 1$ qubits in the F register, for implementing the logical OR gate. We consider the circuit's output is stored in the F_k qubit. In the case of F_k being measured as one, the selected decoder is found and its Enable line is on (meaning that I is set to 1). However, if F_k is measured as zero, the Enable line is turned off (in other words, I is set to 0).

VI. CONCLUSION-FUTURE WORK

This paper presents a parallelizable scheme to efficiently implement decoding and multiplexing operations using a single circuit. The circuit includes a set of oracles which are programmed in such a manner that one input combination is decoded with probability 1 and a set of data selection oracles with parallelism factor $m = \frac{n}{2}$ which guarantees that the final OR result which is required by the multiplexer is implemented much faster. We have compared our strategy to other known strategies, which implement decoders or multiplexers. To have fair comparisons, we have compared our work only against strategies which are expandable to larger number of inputs. The theoretical comparisons have shown that our scheme executes much faster compared to the compared schemes. Although it requires more hardware, this pays off by the high increase in speed. To guarantee its functionality, We have implemented our decoding/multiplexing scheme on a real quantum environment using Qiskit, an open-source framework to program quantum computers.

In the future, we would like to continue our Qiskit experiments in order to find more feasible solutions that reduce the required hardware, without affecting the speed of operation. Also, to take full advantage of quantum logic, we will investigate the possibility of having our circuit operating simultaneously with different input combinations. Also, we will conduct extensive research on the implementation of other combinational circuits like carry look-ahead adders, multipliers and dividers.

REFERENCES

[1] Z. Shan, Y. Zhu, and B. Zhao, "A high-performance compilation strategy for multiplexing quantum control architecture," *Sci. Rep.*, vol. 12, no. 1, p. 7132, May 2022, doi: [10.1038/s41598-022-11154-3](https://doi.org/10.1038/s41598-022-11154-3).

[2] S. Shashikumar, A. Kanchagar, and S. R. Kassa, "Design of improved, 8:1 multiplexer using quantum-dot cellular automata technology," *Int. J. Recent Technol. Eng.*, vol. 9, no. 1, pp. 2659–2662, 2020.

[3] D. K. Kole, J. Dutta, A. Kundu, S. Chatterjee, S. Agarwal, and T. Kisku, "Generalized construction of quantum multiplexers and de-multiplexers using a proposed novel algorithm based on universal Fredkin gate," in *Proc. 6th Int. Symp. Embedded Comput. Syst. Design (ISED)*, Patna, India, Dec. 2016, pp. 82–86, doi: [10.1109/ISED.2016.7977059](https://doi.org/10.1109/ISED.2016.7977059).

[4] P. Aruna and G. Komanapalli, "A quantum cost efficient reversible multiplexer for low power applications," in *Proc. 3rd Int. Conf. Artif. Intell. Signal Process. (AISP)*, VIJAYAWADA, India, Mar. 2023, pp. 1–4, doi: [10.1109/AISP57993.2023.10134940](https://doi.org/10.1109/AISP57993.2023.10134940).

[5] Y. Syamala and A. V. N. Tilak, "Synthesis of multiplexer and demultiplexer circuits using reversible logic," *Int. J. Recent Trends Eng. Technol.*, vol. 4, no. 3, pp. 34–38, Nov. 2010.

[6] P. Das, C. A. Pattison, S. Manne, D. M. Carmean, K. M. Svore, M. Qureshi, and N. Delfosse, "AFS: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Apr. 2022, pp. 259–273, doi: [10.1109/HPCA53966.2022.00027](https://doi.org/10.1109/HPCA53966.2022.00027).

[7] R. Anitha and B. Vijayalakshmi, "Simulation of quantum encoder & decoder with flip bit error correction using reversible quantum gates," in *Proc. Int. Conf. Recent Trends Electr., Control Commun. (RTECC)*, Mar. 2018, pp. 99–102, doi: [10.1109/RTECC.2018.8625693](https://doi.org/10.1109/RTECC.2018.8625693).

[8] M. Bhattacharyya and A. Raina, "A quantum algorithm for syndrome decoding of classical error-correcting linear block codes," in *Proc. IEEE/ACM 7th Symp. Edge Comput. (SEC)*, Seattle, WA, USA, Dec. 2022, pp. 456–461, doi: [10.1109/SEC54971.2022.00069](https://doi.org/10.1109/SEC54971.2022.00069).

[9] S. Hamidi-Rad and J. Kaewell, "Quantum channel decoding," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, Broomfield, CO, USA, Sep. 2022, pp. 847–850, doi: [10.1109/QCE53715.2022.00141](https://doi.org/10.1109/QCE53715.2022.00141).

[10] C. Piveteau and J. M. Renes, "Quantum message-passing algorithm for optimal and efficient decoding," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Espoo, Finland, Jun. 2022, pp. 2225–2230, doi: [10.1109/ISIT50566.2022.9834749](https://doi.org/10.1109/ISIT50566.2022.9834749).

[11] H. Jung, J. Kang, and J. Ha, "Quantum maximum likelihood decoding for linear block codes," in *Proc. Int. Conf. Inf. Commun. Technol. Conver. (ICTC)*, Jeju, South Korea, Oct. 2020, pp. 227–232, doi: [10.1109/ICTC49870.2020.9289350](https://doi.org/10.1109/ICTC49870.2020.9289350).

[12] W. J. Munro, N. L. Piparo, and K. Nemoto, "Quantum multiplexing as a resource saver in quantum networks," in *Proc. IEEE Photon. Soc. Summer Topical Meeting Ser. (SUM)*, Waikoloa, HI, USA, Jul. 2018, p. 41, doi: [10.1109/PHOSST.2018.8456694](https://doi.org/10.1109/PHOSST.2018.8456694).

[13] H. Maity, A. Biswas, A. K. Bhattacharjee, and A. Pal, "The quantum cost optimized design of 2:4 decoder using the new reversible logic block," *Micro Nanosyst.*, vol. 12, no. 3, pp. 146–148, Dec. 2020.

[14] G. Nikhil, B. Sharanya, P. B. Reddy, and R. P. Vidyadhar, "Reversible 2:4 decoder using universal Fredkin gate," in *Proc. 6th Int. Conf. Commun. Electron. Syst. (ICES)*, Coimbatore, India, Jul. 2021, pp. 173–176, doi: [10.1109/ICES51350.2021.9489226](https://doi.org/10.1109/ICES51350.2021.9489226).

[15] N. J. Lisa and H. M. H. Babu, "A compact realization of a reversible quantum n-to-2ⁿ decoder," in *Proc. IEEE 4th Int. Conf. Electron. Inf. Emergency Commun.*, Beijing, China, Nov. 2013, pp. 90–93, doi: [10.1109/ICEIEC.2013.6835461](https://doi.org/10.1109/ICEIEC.2013.6835461).

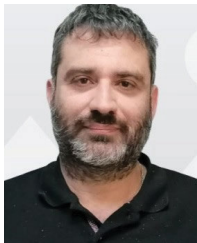
[16] H. M. Pooja, M. G. Nayana, and C. L. Triveni, "Design and implementation of encoder/decoder quantum circuits using decimal to binary conversion method and vice versa," in *Proc. IEEE Int. Conf. Electron., Comput. Commun. Technol. (CONECCT)*, Bangalore, India, Jul. 2021, pp. 1–5, doi: [10.1109/CONECCT52877.2021.9622719](https://doi.org/10.1109/CONECCT52877.2021.9622719).

[17] S. M. A. Mirzadeh, M. M. E. Kouchak, and M. M. Panahi, "A novel design of quantum 3:8 decoder circuit using reversible logic for improvement in key quantum circuit design parameters," in *Proc. 26th Int. Comput. Conf., Comput. Soc. Iran (CSICC)*, Tehran, Iran, Mar. 2021, pp. 1–5, doi: [10.1109/CSICC52343.2021.9420575](https://doi.org/10.1109/CSICC52343.2021.9420575).

[18] N. Mahmud, M. J. Jeng, Md. A. I. Nobel, M. Chaudhary, S. M. I. U. Islam, D. Levy, and E. El-Araby, "Efficient data encoding and decoding for quantum computing," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, Broomfield, CO, USA, Sep. 2022, pp. 765–768, doi: [10.1109/QCE53715.2022.00110](https://doi.org/10.1109/QCE53715.2022.00110).

[19] M. A. Mohd Izhar, Z. Babar, H. V. Nguyen, P. Botsinis, D. Alanis, D. Chandar, S. X. Ng, and L. Hanzo, "Quantum turbo decoding for quantum channels exhibiting memory," *IEEE Access*, vol. 6, pp. 12369–12381, 2018, doi: [10.1109/ACCESS.2018.2808373](https://doi.org/10.1109/ACCESS.2018.2808373).

- [20] W. J. Munro, N. L. Phipps, and K. Nemoto, "Quantum multiplexing," in *Proc. Conf. Lasers Electro-Optics (CLEO)*, San Jose, CA, USA, May 2019, pp. 1–2, doi: [10.1364/CLEO_QELS.2019.FTh4A.1](https://doi.org/10.1364/CLEO_QELS.2019.FTh4A.1).
- [21] J. Gil-Lopez, L. Serino, M. Santandrea, W. Ridder, V. Ansari, B. Brecht, and C. Silberhorn, "The multi-output quantum pulse gate: A novel high-dimensional QKD decoder," in *Proc. Conf. Lasers Electro-Optics Eur. Eur. Quantum Electron. Conf.*, Munich, Germany, Jun. 2021, pp. 1–11, doi: [10.1109/CLEO/Europe-EQEC52157.2021.9541915](https://doi.org/10.1109/CLEO/Europe-EQEC52157.2021.9541915).
- [22] M. Selim, "Quantum cost optimization for reversible sequential circuit," *Int. J. Adv. Comput. Sci. Appl.*, vol. 4, no. 12, pp. 15–21, 2013.
- [23] Accessed: Feb. 10, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Quantum_logic_gate
- [24] Accessed: Feb. 10, 2023. [Online]. Available: <https://www.quantum-inspire.com/kbase/cnot/>
- [25] Accessed: Feb. 10, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Controlled_NOT_gate
- [26] Accessed: Feb. 10, 2023. [Online]. Available: https://quantum-computing.ibm.com/composer/docs/idx/operations_glossary/
- [27] Accessed: Feb. 10, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Toffoli_gate



STAVROS SOURAVLAS (Member, IEEE) is currently an Assistant Professor in computer architecture and digital logic design with the Department of Applied Informatics, School of Information Sciences, University of Macedonia, where he joined, in 2014. His research interests include computer architecture and performance evaluation, parallel and distributed systems, big data stream scheduling, cloud computing, systems modeling, and simulation.



MARIA ORFANAKI is currently pursuing the degree with the Department of Applied Informatics, University of Macedonia. She received a scholarship from IBM Quantum to attend "Introduction to Quantum Computing." Her research interests include optimization algorithms and embedded systems. She is coordinating the university's robotics team and she is a member at IEEE UOM Student Branch and EESTEC LC Thessaloniki.



EVANGELIA GIROUSI is currently pursuing the degree with the Department of Applied Informatics, School of Information Sciences, University of Macedonia. Her research interests include computer architecture, quantum computing, and artificial intelligence. She is a member of the IEEE Student Branch of the University of Macedonia and a member of the university's robotics team.

...